Lesson 10 Teacher's Checklist / Prep Guide

Complete before class

VIDEO GAME CREATION

This Week: Closing the Loop

Since Lesson 8, winning or losing has just meant a message appearing while the game kept quietly running underneath. This week finally closes that loop: the game actually stops when it's over, and pressing R brings it back to a genuinely fresh start - not a reload, not a re-launch, an in-place reset built entirely from code students already trust.

The central idea this week is reuse, one more time: instead of writing separate 'set up a new game' code and 'reset the game' code, every engine's design this week factors the ORIGINAL one-time setup (that's been running since Lesson 5) into a single reusable routine, called once at first load AND again every time R is pressed. This is the clearest, most direct example yet of the reuse principle threaded through this whole course.

- ☐ Read through each engine's Handout once before class - every engine involves a genuine refactor this week (moving existing setup code into a new reusable form), not just new additions.

A game_active Flag, Introduced This Week

Every engine gets a new boolean-ish flag (game_active, GameActive, or similar) that gates whether gameplay logic still applies. Once a round ends (win or lose), this flag flips off - existing win/lose-check code stops changing score or lives further, and (where applicable) enemy movement or player input freezes. Pressing R while the flag is off calls the reset routine, which flips it back on as its last step.

001 Game Creator: Verified Mechanisms

This week's 001 Game Creator design was researched and verified against the official wiki before writing: the restart key uses an Input Trigger (the same underlying mechanism already used since Lesson 2 for movement and shooting), and the reusable reset routine uses a Custom Event (a genuine, documented reusable-subroutine feature - 001gamecreator.com/wiki/customevents/). A built-in "Restart Game" action does exist in the engine, but this lesson deliberately teaches the manual, verified reset approach instead, since it's better pedagogy and its exact reset behavior isn't documented clearly enough to teach with confidence.

- ☐ If you have Windows access before class, walk through creating one Custom Event and one Input Trigger yourself - both are new concepts for this course, even for an instructor comfortable with 001 Game Creator's other patterns.

TIC-80: Verified Key Code

TIC-80's restart key detection uses keyp(18) - confirmed directly against the engine's own GitHub wiki key code table (keycode 18 = R) and the keyp() function's documented edge-trigger behavior (fires true only on the frame a key is first pressed, not while held). This was verified this week given this engine's history of needing extra care with exact API details.

Self-Contained Lesson Assets

No new assets this week - restart flow is entirely code/logic.

Continuity From Lesson 9

- ☐ Remind students to open their own [YourName]Game project from Lesson 9, not create a new one.
- ☐ If a student's Lesson 9 project is lost or broken, have a pre-built "Lesson 9 complete" starter available per engine (from the Lesson 9 Teacher's Keys) so they aren't blocked.

Engine Verification

- ☐ Confirm the demo laptop's Lesson 6-9 exercises still work for all 8 engines.
- ☐ Test a FULL round-trip on every engine before class: play to a win, press R, confirm a clean restart; separately play to a loss, press R, confirm a clean restart. Both directions matter.
- ☐ 001 Game Creator (Windows demo laptop): walk through the Custom Event + Input Trigger combination once, and specifically confirm the 5 pre-placed enemy copies revive and reposition correctly on reset.
- ☐ Godot / Unity / GameMaker / GDevelop / Unreal: confirm any enemies still on screen at game-over are properly cleared before the fresh set spawns on restart - a common thing to miss when first building this.

Materials

- ☐ Print or queue digital copies of all 8 Student Handouts (or confirm students can access their engine's version electronically).
- ☐ Have the 8 Teacher's Keys on hand (instructor reference only - do not distribute) in case a student needs a worked example beyond Level 1.
- ☐ Bring: demo laptop, projector cable/adapter, handout copies.

Known Gotchas This Week

- The most likely mistake across every engine: forgetting to clear/destroy leftover enemies before respawning on restart, resulting in double the enemies (or a wrong enemies_remaining count) after the first restart.
- A close second: pressing R while the game is still active accidentally triggers a restart, because the game_active check was left out of the restart-key logic. Test this explicitly - R should do nothing mid-game.
- Score/lives sometimes still change after game-over because the win/lose-check code wasn't actually wrapped in a game_active check - only the display was frozen, not the underlying logic. This is a subtle miss worth testing for directly, not just eyeballing the screen.

- This is a satisfying lesson to finish - encourage a full playthrough demo (win, restart, lose, restart) as part of the end-of-lesson share-out, since it's the first time the game truly feels replayable.